

CS360 Introduction to Database

The Relational Model of Data

Min-Soo Kim

School of Computing, KAIST

What is a data model?

- Data model: notation for **describing** data or information
 - description generally consists of three parts
- Structure of data
 - e.g., **a set of tables of tuples and columns** (in relational model)
 - e.g., graph(s) of vertices and edges (in graph model)
 - **conceptual** model rather than **physical** data model
- Operations on data
 - **queries**: e.g., retrieve a subtable satisfying a specific condition
 - **modifications**: e.g., update a column value of a tuple
- Constraints on data
 - e.g., type and range for the age column: integer of [0, 120]

Relational model

- Proper physical implementation for a relation
 - must consider the need to access relations of very large size that are resident on disk (not in main memory)
- Operations: relational algebra
 - e.g., all the rows where the **genre** is “comedy”
- Constraints: no two movies have the same title and year
- e.g., **Movies** relation
 - each row represents a movie
 - each column represents a property of movies

title	year	length	genre
Gone With the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne’s World	1992	95	comedy

Basics of the relational model

- Row = tuple = record = object
- # of rows : cardinality
- Column = attribute = field = feature
- # of columns : degree = dimension = arity
- Schema (for a relation)
 - name of the relation + set of attributes for the relation
 - e.g., Movies (title, year, length, genre)
- Database schema
 - set of schemas for the relations of a database
- Tuples
 - e.g., (Gone With the Wind, 1939, 231, drama)

- **Domains** (specific fixed type)
 - **SQL-92** (or **SQL2**, **SQL:1992**) supports the **relational** model
 - SQL-92 requires that each column value of each tuple be atomic (i.e., elementary type)
 - e.g., **Movies** (title: string, year: integer, length: integer, genre: string)
 - **SQL-99** (or **SQL3**, **SQL:1999**) supports the **object-oriented** relational model
 - SQL-99 allows each column value to be a structured or user-defined type
 - e.g., **Customers** (name: string, age: integer, address: Address)
 - e.g., **Address** (country: string, state: string, city: string, street: string, zip: integer)
- c.f., **SQL:2023** (in 2024) supports **graph** query processing

- Equivalent representations of a relation
 - a relation: a **set** of tuples (rather than a list of tuples)
 - we can reorder the attributes as we choose, in theory

title	year	length	genre
Gone With the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne's World	1992	95	comedy

the same
(six possible orders)

title	year	length	genre
Star Wars	1977	124	sciFi
Wayne's World	1992	95	comedy
Gone With the Wind	1939	231	drama

the same,
can be reordered in theory
(practically difficult)

year	genre	title	length
1939	drama	Gone With the Wind	231
1977	sciFi	Star Wars	124
1992	comedy	Wayne's World	95

Relation instances

- A relation is not static; rather **change over time**
 - insert new **tuples**
 - update the existing **tuples** (i.e., some column values)
 - delete the existing **tuples**
- Less common for the **schema** of a relation to change
 - we might want to add or delete attributes
 - changing schema is **possible, but very expensive**
- A set of tuples for a relation is called as an **instance**

Keys of relations

- A set of attributes forms a **key** for a relation
 - if we do **not allow two tuples** in a relation instance to **have the same values** in all the attributes of the key
 - e.g., two attributes title and year
 - c.f., face or fingerprint for distinguishing a person from others
 - e.g., Movies (title, year, length, genre)
 - many real-world databases use artificial keys
 - e.g., employee ID, student ID, social-security number (SSN), ...

Example database schema about movies

Movies (

title: string,
year: integer,
length: integer,
genre: string,
studioName: string,
producerC#: integer

)

MovieStar (

name: string,
address: string,
gender: char,
birthdate: date

)

StarsIn (

movieTitle: string,
movieYear: integer,
starName: string

)

MovieExec (

name: string
address: string,
cert#: integer,
netWorth: integer

)

Studio (

name: string,
address: string,
presC#: integer

)

Defining a relation schema in SQL

- SQL (pronounced “sequel”)
 - standard language used to describe and manipulate relational databases
- Two aspects to SQL:
 - DDL for declaring database schema
 - DML for querying database and for modifying the database
- Relations in a SQL statement
 - **stored relations** in database (i.e., tables)
 - **views** : relations computed by queries
 - not stored, but are constructed on-the-fly when needed
 - **temporary tables** : constructed by query processor temporarily
 - thrown away and NOT stored

- Primitive data types supported by SQL systems
 - character string of fixed length : `CHAR (n)`
 - with padding
 - character string of varying length : `VARCHAR (n)`
 - without padding, using an endmarker or string-length
 - bit string of fixed length : `BIT (n)`
 - bit string of varying length : `BIT VARYING (n)`
 - `BOOLEAN`
 - `INT` or `INTEGER`, `SHORTINT`
 - `FLOAT` or `REAL`, `DOUBLE PRECISION`
 - `DECIMAL (n, d)` : `n` decimal digits
 - with the decimal point assumed to be `d` positions from the right
 - e.g., 123.45 is a possible value of type `DECIMAL(5,2)`
 - `DATE`, `TIME`

- Simple table declarations: **CREATE TABLE** followed by
 - the name of the relation
 - a parenthesized comma-separated list of the attribute names and their types

- e.g.,

```
CREATE TABLE Movies (  
    title          CHAR(100),  
    year          INT,  
    length        INT,  
    genre         CHAR(10),  
    studioName    CHAR(30),  
    producerC#   INT  
);
```

```
CREATE TABLE MovieStar (  
    name          CHAR(30),  
    address       VARCHAR(255),  
    gender        CHAR(1),  
    birthdate     Date  
);
```

Modifying relation schemas

- Delete an existing relation: **DROP TABLE**
 - e.g., `DROP TABLE Movies;`
- Modify the schema of an existing relation: **ALTER TABLE**
 - ADD followed by an attribute name and its data type
 - e.g., `ALTER TABLE MovieStar ADD phone CHAR(16);`
 - DROP followed by an attribute name
 - e.g., `ALTER TABLE MovieStar DROP birthdate;`

Default values

- We sometimes do not have values for all attributes
 - e.g., when we add a column to a relation schema, the existing tuples do not have a known value
 - we would prefer to use default value instead of **NULL**
 - e.g., we might set a default value ? for unknown **gender**
 - e.g., we might set the earliest possible date as the default for unknown **birthdate**

```
gender      CHAR(1) DEFAULT '?',  
birthdate  DATE      DEFAULT DATE '0000-00-00'
```

- e.g., default value for new attribute **phone**

```
ALTER TABLE MovieStar ADD phone CHAR(16) DEFAULT 'unlisted';
```

Declaring keys

- Declare in **CREATE TABLE**
 - add keyness to the line declaring an attribute
 - use a separate definition of the key (for multiple attributes)
- Two kinds of declarations
 - **PRIMARY KEY**: NULL is not permitted as a key value
 - **UNIQUE**: NULL is permitted as a key value

```
CREATE TABLE MovieStar (  
    name          CHAR(30) UNIQUE,  
    address       VARCHAR(255),  
    gender        CHAR(1),  
    birthdate     Date  
);  
  
CREATE TABLE Movies (  
    title          CHAR(100),  
    year          INT,  
    length        INT,  
    genre         CHAR(10),  
    studioName    CHAR(30),  
    producerC#    INT,  
    PRIMARY KEY (title, year)  
);
```

Algebraic query language

- Relational algebra (special algebra for relational model)
 - simple but powerful ways to construct new relations from given relations
 - sub-relation of the constructed relations is answered to queries
- SQL incorporates relational algebra at its center
 - SQL is “syntactically sugared” expressions of relational algebra
 - SQL query is translated into relational algebra or a very similar internal representation (i.e., query plan)

Why do we need a special query languages?

- SQL or relational algebra is **NOT** a general programming language
 - there are a lot of computations that one can perform in C, but cannot perform in relational algebra
 - e.g., drawing pixels, transferring data, determining whether the number is odd or even
 - but, **SQL is powerful enough to compute a result under the relational model**
- Two huge rewards from less powerfulness
 - ease of programming (done by **users**)
 - ability of the compiler to produce highly optimized code (done by **DBMS**)

What is an algebra?

- In general, an algebra consists of operators and atomic operands
- e.g., algebra of arithmetic
 - atomic operands: variable x , constant 15
 - operators: addition, subtraction, multiplication, and division
- Any algebra allows us to build expressions
 - usually, use parentheses to group operators and operands
 - e.g., $((x + 7) / (y - 3)) + x$
- Relational algebra
 - atomic operands: relations
 - operators: selection, projection, product, union, difference, ...

Overview of operators of relational algebra

- Usual **set** operations
 - union, intersection, and difference
- Operations that **remove parts of a relation**
 - **selection** : eliminates some rows (tuples)
 - **projection** : eliminates some columns
- Operations that **combine the tuples of two relations**
 - **Cartesian product** : pairs the tuples of two relations in all possible ways
 - **join** : selectively pairs the tuples of two relations
- **Renaming** operation
 - changes relation schema (name of relation, names of attributes)

Set operations on relations

- Most common operations on sets:
 - union: $R \cup S$ (set of tuples that are in R or S or both)
 - intersection: $R \cap S$ (set of tuples that are in both R and S)
 - difference: $R - S$ (set of tuples that are in R but not in S)
 - note: $R - S$ is different from $S - R$
- Conditions on R and S for set operations on relations
 - R and S **must have the same schema (i.e., names and types)**
 - if R and S have the same # of attributes with the same types, but use different names, we may use the **renaming operator**
 - columns of R and S **must be ordered in the same attribute order**

relation R	name	address	gender	birthdate
	Carrie Fisher	123 Maple St., Hollywood	F	9/9/99
	Makrk Hamill	456 Oak Rd., Brentwood	M	8/8/88

relation S	name	address	gender	birthdate
	Carrie Fisher	123 Maple St., Hollywood	F	9/9/99
	Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

relation R

name	address	gender	birthdate
Carrie Fisher	123 Maple St., Hollywood	F	9/9/99
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88

relation S

name	address	gender	birthdate
Carrie Fisher	123 Maple St., Hollywood	F	9/9/99
Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

• $R \cup S$

name	address	gender	birthdate
Carrie Fisher	123 Maple St., Hollywood	F	9/9/99
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88
Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

• $R \cap S$

name	address	gender	birthdate
Carrie Fisher	123 Maple St., Hollywood	F	9/9/99

• $R - S$

name	address	gender	birthdate
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88

Projection

- Produce a relation that has a subset of R's columns
 - $\pi_{A_1, A_2, \dots, A_n}(R)$: a relation that has attributes $A_1, A_2, \dots, A_n$ of R

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345
Galaxy Quest	1999	104	comedy	DeamWorks	67890
Wayne's World	1992	95	comedy	Paramount	99999

$\pi_{\text{title, year, length}}(\text{Movies})$

title	year	length
Star Wars	1977	124
Galaxy Quest	1999	104
Wayne's World	1992	95

$\pi_{\text{genre}}(\text{Movies})$

genre
sciFi
comedy

Selection

- Produce a relation that has a subset of R's tuples
 - $\sigma_C(R)$: a set of tuples of R that satisfies the condition C
 - the resulting relation's schema is the same with R's schema

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345
Galaxy Quest	1999	104	comedy	DeamWorks	67890
Wayne's World	1992	95	comedy	Paramount	99999

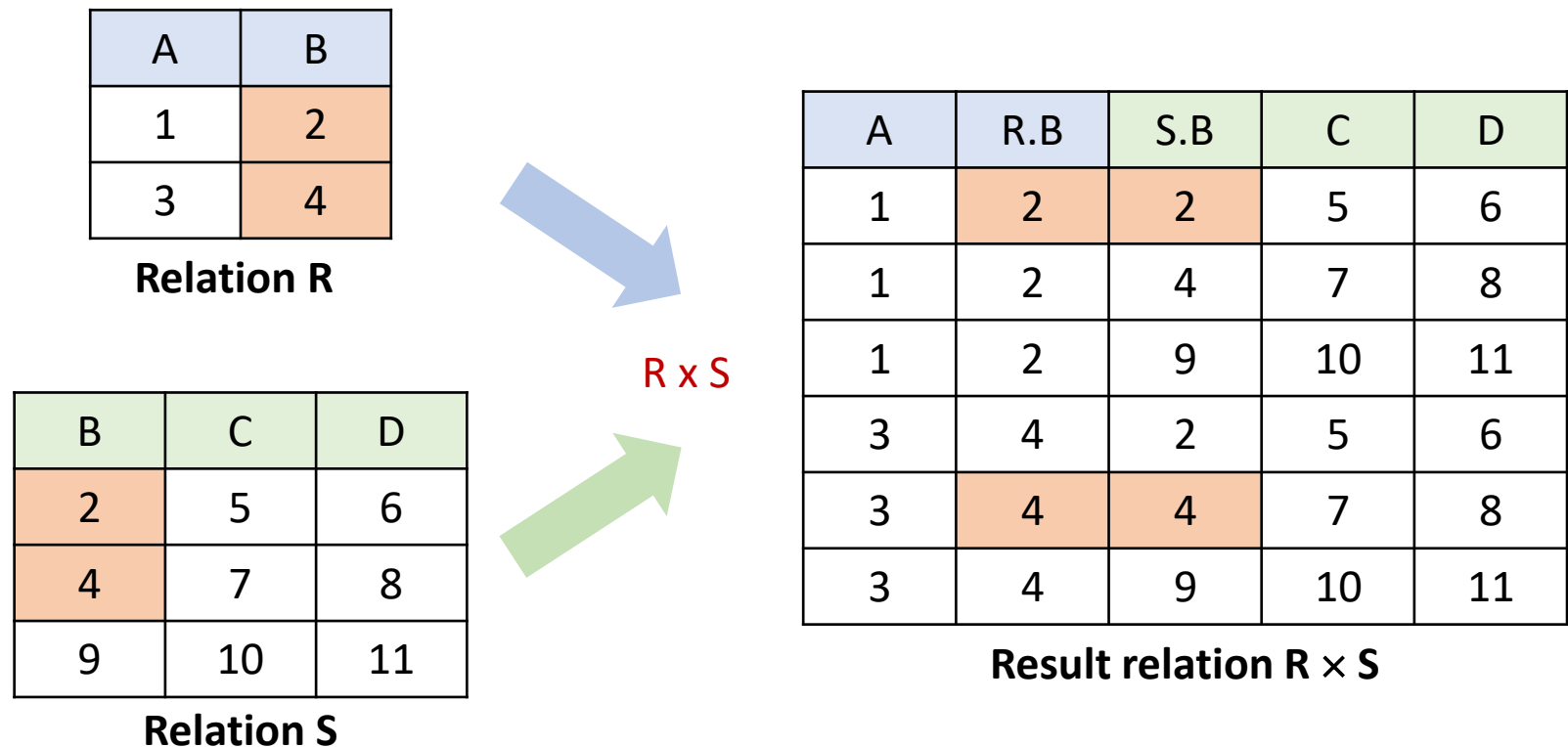


$\sigma_{\text{length} \geq 100 \text{ AND studioName='Fox'}}(\text{Movies})$

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345

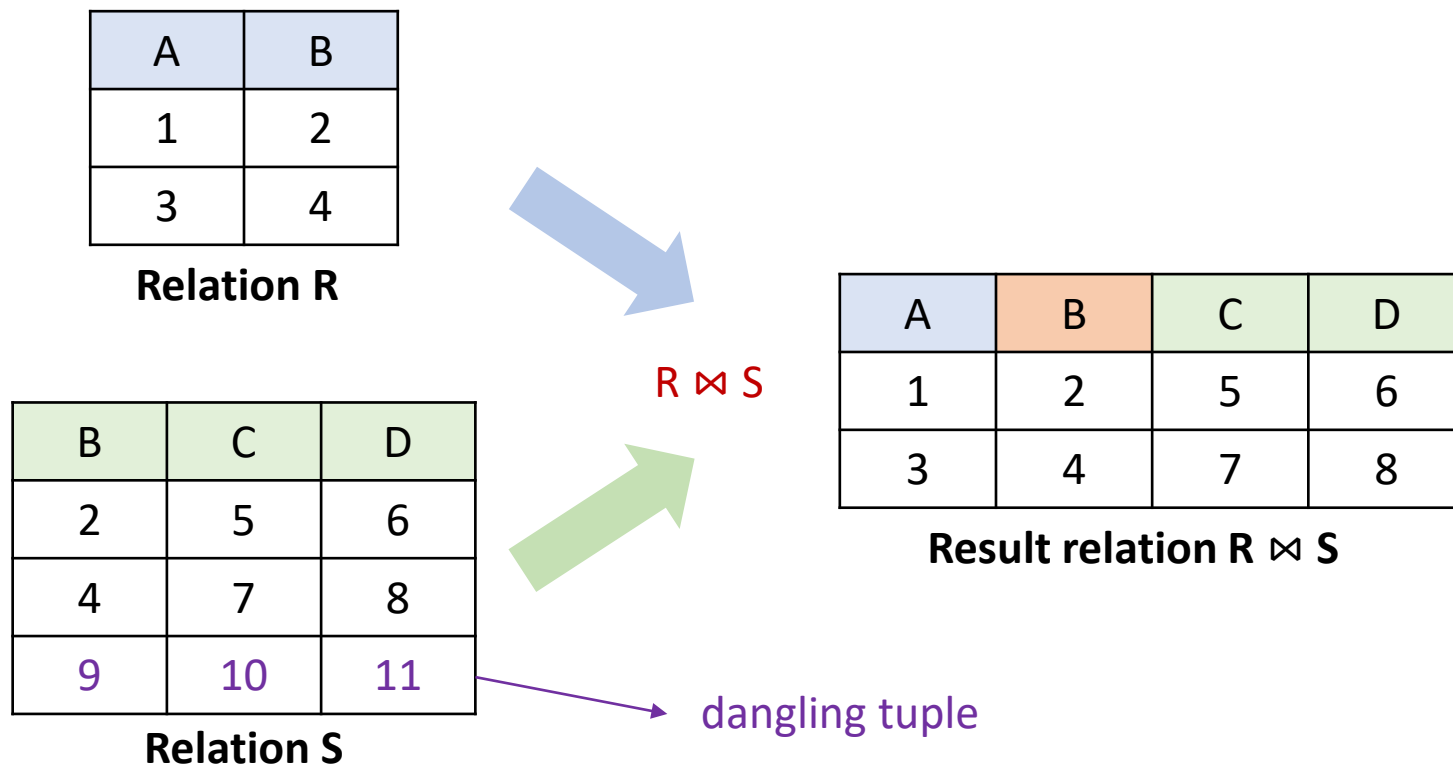
Cartesian product (cross-product, product)

- $R \times S$: a relation that has all possible pairs between R and S
 - the resulting relation's schema is the same with $R + S$
 - we use R.A and S.A to disambiguate an attribute A in R and S



Natural joins

- $R \bowtie S$: a set of pairs of tuples $r \in R$ and $s \in S$ that agree on $A_1, A_2, \dots, A_n$, the common attributes of R and S
 - joined tuple : a tuple of the resulting relation $R \bowtie S$
 - the schema of $R \bowtie S$ is the same with $(R + S - \{A_1, A_2, \dots, A_n\})$



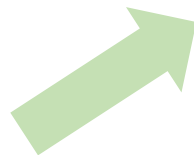
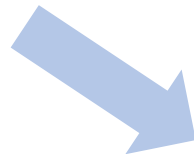
- Another example

A	B	C
1	2	3
6	7	8
9	7	8

Relation U

B	C	D
2	3	4
2	3	5
7	8	10

Relation V



$U \bowtie V$

A	B	C	D
1	2	3	4
1	2	3	5
6	7	8	10
9	7	8	10

Resulting relation $U \bowtie V$

Theta-joins

- Historically, θ (theta) refers to an arbitrary condition
- $R \bowtie_C S$: product forcing a condition C when pairing tuples
 - take the product of R and S
 - select the tuples that satisfy C from the product

A	B	C
1	2	3
6	7	8
9	7	8

Relation U

B	C	D
2	3	4
2	3	5
7	8	10

Relation V

$U \bowtie_{A < D} V$



A	U.B	U.C	V.B	V.C	D
1	2	3	2	3	4
1	2	3	2	3	5
1	2	3	7	8	10
6	7	8	7	8	10
9	7	8	7	8	10

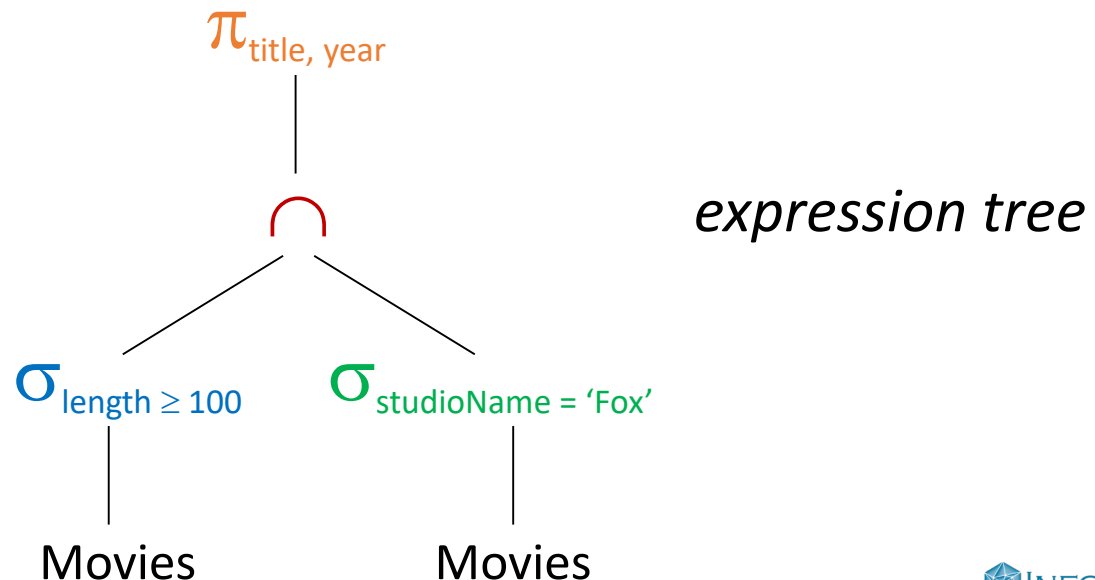


A	U.B	U.C	V.B	V.C	D
1	2	3	7	8	10

$U \bowtie_{A < D \text{ AND } U.B \neq V.B} V$

Combining operations to form queries

- Query: “What are the titles and years of movies made by Fox that are at least 100 minutes long?”
 - ① select those Movies tuples that have length ≥ 100
 - ② select those Movies tuples that have studioName = ‘Fox’
 - ③ compute the intersection of (1) and (2)
 - ④ project the relation from (3) onto attributes *title* and *year*



- The steps for query processing as an **expression tree**
 - expression tree are evaluated **bottom-up**
 - by applying the operator at an interior node to the arguments
 - arguments: the results of its children
- Alternatively, the same expression in a **linear notation**

$$\pi_{\text{title, year}} (\sigma_{\text{length} \geq 100} (\text{Movies}) \cap \sigma_{\text{studioName} = \text{'Fox'}} (\text{Movies}))$$

- There is often more than one relational algebra expression
 - an **equivalent** form of the query
 - e.g., replacing the intersection by logical AND

$$\pi_{\text{title, year}} (\sigma_{\text{length} \geq 100 \text{ AND studioName} = \text{'Fox'}} (\text{Movies}))$$

Naming and renaming

- Used for controlling the names of attributes of the relation
 - that is constructed by applying relational algebra operations
- $\rho_{S(A_1, A_2, \dots, A_n)}(R)$
 - S : new name of the relation
 - $A_1, A_2, \dots, A_n$: new names of the attributes
 - S has exactly the same tuples as R

Relation R

A	B
1	2
3	4

Relation S

B	C	D
2	5	6
4	7	8
9	10	11

$$R \times \rho_{S(X, C, D)}(S)$$

A	B	X	C	D
1	2	2	5	6
1	2	4	7	8
1	2	9	10	11
3	4	2	5	6
3	4	4	7	8
3	4	9	10	11

alternatively, $\rho_{RS(A, B, X, C, D)}(R \times S)$

Relationships among operations

- Some operations can be expressed in terms of others

$$R \cap S = R - (R - S)$$

- Theta-join can be expressed

$$R \bowtie_C S = \sigma_C(R \times S)$$

- Natural join can be expressed

$$R \bowtie S = \pi_L(\sigma_C(R \times S))$$

- with a condition C of the form

$$R.A_1 = S.A_1 \text{ AND } R.A_2 = S.A_2 \text{ AND } \dots \text{ AND } R.A_n = S.A_n$$

- where $A_1, A_2, \dots, A_n$ are all the attributes appearing in both R and S
 - L: the list of attributes in the schema R followed by those attributes in S that are not in the schema of R

Example (1)

$$U \bowtie V = \pi_{A, U.B, U.C, D}(\sigma_{U.B=V.B \text{ AND } U.C=V.C}(U \times V))$$

A	B	C
1	2	3
6	7	8
9	7	8

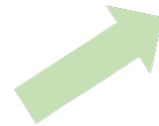
Relation U

B	C	D
2	3	4
2	3	5
7	8	10

Relation V



$U \bowtie V$



A	B	C	D
1	2	3	4
1	2	3	5
6	7	8	10
9	7	8	10

Resulting relation $U \bowtie V$

Example (2)

$$U \bowtie_{A < D \text{ AND } U.B \neq V.B} V \longrightarrow \sigma_{A < D \text{ AND } U.B \neq V.B}(U \times V)$$

Six basic operations

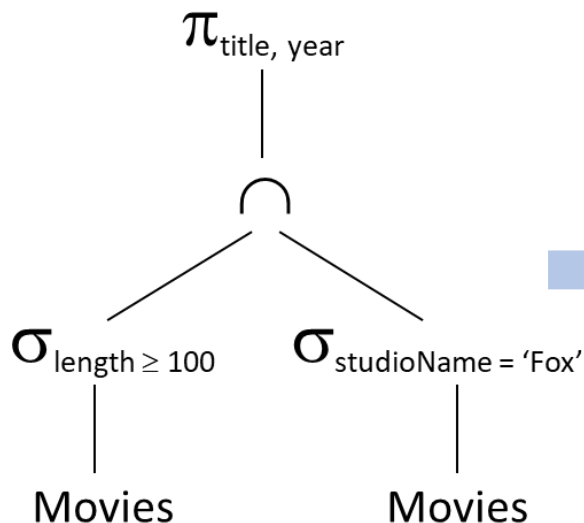
- Intersection, theta-join, and natural join: redundant
 - they can be expressed using other operations
- Six basic operations
 - union, difference, selection, projection, product, and renaming
 - they form an independent set
 - none of which can be written in terms of the other operations

Linear notation for algebraic expressions

- Alternative notation

- invent names for **temporary relations** that correspond to the interior nodes of the expression tree
- write a sequence of assignments that create a value for each

- e.g.,



$$\pi_{\text{title, year}} (\sigma_{\text{length} \geq 100} (\text{Movies}) \cap \sigma_{\text{studioName} = \text{'Fox'}} (\text{Movies}))$$



$$\left[\begin{array}{l} R(t, y, l, i, s, p) := \sigma_{\text{length} \geq 100} (\text{Movies}) \\ S(t, y, l, i, s, p) := \sigma_{\text{studioName} = \text{'Fox'}} (\text{Movies}) \\ T(t, y, l, i, s, p) := R \cap S \\ \text{Answer}(\text{title, year}) := \pi_{t,y}(T) \end{array} \right.$$

- Each assignment statement consists of
 - ① relation name and its list of attributes (on left left)
 - “Answer” will be used conventionally for the result of the final step
 - ② assignment symbol $:=$
 - ③ any algebraic expression (on the right)
 - usually one operator per assignment
 - each interior node usually gets its own assignment statement
 - but, it is also possible to combine several operations in one right side

$R(t, y, l, i, s, p) := \sigma_{\text{length} \geq 100}(\text{Movies})$
 $S(t, y, l, i, s, p) := \sigma_{\text{studioName} = \text{'Fox'}}(\text{Movies})$
 $T(t, y, l, i, s, p) := R \cap S$
 $\text{Answer}(\text{title}, \text{year}) := \pi_{t,y}(T)$



$R(t, y, l, i, s, p) := \sigma_{\text{length} \geq 100}(\text{Movies})$
 $S(t, y, l, i, s, p) := \sigma_{\text{studioName} = \text{'Fox'}}(\text{Movies})$
 $\text{Answer}(\text{title}, \text{year}) := \pi_{t,y}(R \cap S)$

Constraints on relations

- Data model can restrict the data that may be stored in DB
- We have seen only **key constraints**
 - no two tuples agree on the attributes of key
- **Referential integrity constraints**
- Other constraints

Example database schema about movies (revisited)

Movies (

title: string,
year: integer,
length: integer,
genre: string,
studioName: string,
producerC#: integer

)

MovieStar (

name: string,
address: string,
gender: char,
birthdate: date

)

StarsIn (

movieTitle: string,
movieYear: integer,
starName: string

)

MovieExec (

name: string
address: string,
cert#: integer,
netWorth: integer

)

Studio (

name: string,
address: string,
presC#: integer

)

Referential integrity constraints

- A value appearing in one column of one relation also appears in some other column of related relations
- e.g.,
 - we expect that a *starName* column value of *StarsIn* appears as a *name* column value of *MovieStar*

$$\pi_{starName}(StarsIn) \subseteq \pi_{name}(MovieStar)$$

- we expect that a *movieYear* (or *movieTitle*) column value of *StarsIn* appears as a *year* (or *title*) column value of *Movies*

$$\pi_{movieYear}(StarsIn) \subseteq \pi_{year}(Movies)$$

$$\pi_{movieTitle}(StarsIn) \subseteq \pi_{title}(Movies)$$

Another example

- We reasonably assume that
 - **producer** of every movie would have to appear in *MovieExec*
 - if not, there is something wrong
 - we can express this constraint by

$$\pi_{\text{producerC\#}}(\text{Movies}) \subseteq \pi_{\text{cert\#}}(\text{MovieExec})$$

Movies (		MovieExec (
<u>title</u> : string,		name: string
<u>year</u> : integer,		address: string,
length: integer,		<u>cert#</u> : integer,
genre: string,		netWorth: integer
studioName: string,		)
producerC# : integer	←	
)		

Thank you

Q & A